

Table of contents

Introduction to Bash	3
pwd: Find out where we are	3
ls: List the contents of a directory	3
The structure of a command	4
Getting help	5
mkdir: Make new folders	5
touch: Starting your notebook	7
Writing the notebook in Markdown	8
cd: Move around folders	9
wget: Download data	13
cp: Copying data	14
mv: Moving (and renaming) data	15
tar: Work with tar files	16
rm: Remove files and folders	17
Wildcards	18
I/O redirection to new files	23
Exploring the content of text files	24
head: View the first rows of a file	24
tail: View the last rows of a file	24
less: View the full file	25
wc: Count things	26
Pipes	26
cut: Extract elements from strings	27
sort and uniq: Create unique lists	29
cat: Read and combine files	30
Exploring the content of compressed fastq files	32
gzip: (De)compress files	32
zcat: Decompress and print to screen	33
grep: Find patterns in data	34
wc: Count things II	35
echo and variables: Storing information	39
For-loops	40
Step 1: Check what the loop works on	40

Step 2: Add the real command	41
>>: Collecting the loop output in one file	42
bash : Running commands from a script	47
Wrapping up day 1	49

Introduction to Bash

pwd: Find out where we are

To get started, open your terminal and get yourself oriented by typing your first command, `pwd`, and then pressing enter:

```
pwd
```

The command `pwd` stands for *print working directory*. The **working directory** is the folder your shell is operating on right now, so `pwd` tells you where you currently are in the file system. You should see something like this:

```
/Users/YourUserName
```

When you open a new terminal, you start in what is called your **home directory**. This is your personal folder on the computer.

What the path to your home directory looks like depends on your system:

- **Mac:** `/Users/YourUserName`
- **MobaXterm (Windows):** `/home/mobaxterm`
- **WSL2 (Windows) and Linux:** `/home/YourUserName`

In this tutorial, we work inside the home directory. That way, the commands are the same for everybody.

ls: List the contents of a directory

Now that you know where you are, let's find out what is inside that location, i.e. what files and folders can be found there. The command `ls` (short for *list*) shows the files and folders in your current directory. Type the following and press enter:

```
ls
```

In my case this returns something like this (and this will look different for your current directory):

The colors and formatting depend on your terminal settings, but typically:

- Folders (directories) appear in one color (in my example above: green)
- Files appear in another (in my example above: bold white)

If your directory contains many items, the output can quickly become overwhelming. To make sense of it, we can use options and arguments to control how commands behave.

The structure of a command

A command generally has three parts:

- A **command name**: the program you want to run, e.g. `ls`
- An **option** (or **flag**): a way to change how the command behaves, e.g. `-l` (long format). Options usually start with a `-`
- An **argument**: the input the command works on, e.g. a file or folder. Some commands work without an argument

Try the following command in your current directory to list (`ls`) the contents of the current folder and show details in long format (`-l`):

```
ls -l
```

After running this, we see our files and folders again but as a more detailed list. In the example below you can now see additional information about who is allowed to read and change the files (the access modes), who owns them, how large the files are, when they were last modified and of course their name:

Tip: Using ls in practice

Throughout this tutorial, you'll notice that we use `ls` a lot. There is a reason for that. When working with data, sanity checks are essential, because it is easy to make mistakes, overwrite files, or lose track of where things are. Simple commands like `ls`, `pwd`, `wc` or `grep` help you check what is happening to your files at every step of an analysis.

These habits are not just for beginners, bioinformaticians rely on them constantly.

Getting help

At some point, you'll want to know what options a command has or how it works. In this case, first check if a **manual page** (or *man page*) is available for the command by typing **man** followed by the command name:

```
man ls
```

This opens the manual entry for the command `ls`. You can move through it using:

- the `↑` / `↓` arrow keys, or the space bar to move down one page
- `b` to move back up one page
- `q` to quit the manual

Not all programs come with such a manual. Depending on the program, there are a few common ways to get help that you can try:

- `man ls`
- `ls --help`
- `ls -h`

mkdir: Make new folders

Before we start moving around, let's first learn how to create new folders (also called directories). This is something we will do often, for example to keep raw data, results and scripts organized in separate places. The command we use for that is **mkdir**, which stands for *make directory*.

We use **mkdir** to create a project folder called **data_analysis** for this tutorial. Don't worry about the `cd ~` in the first line yet, we cover moving around in the next section. For now, it is enough to know that it takes you to your home directory.

```
# Move into the home directory
cd ~
```

```
# Create a new folder called data_analysis
mkdir data_analysis

# Check that the folder was created
ls
```

You should see a new folder called **data_analysis** in the list. We will use this folder as our project folder for all exercises in this tutorial.

You can also look at the same folder in the file browser you know from Windows or Mac. The command below opens the folder you are in right now. The dot (.) is short for “the current folder”:

```
# Windows (MobaXterm or WSL2): open the current folder in File
↳ Explorer
explorer.exe .

# Mac: open the current folder in Finder
open .
```

Use the command for your own system. You should see a window with your home directory and the new **data_analysis** folder in it. This is useful later, for example to open a report that you made with the command line.

! MobaXterm users: did you choose a permanent folder?

If `explorer.exe .` opens a folder in a temporary location, MobaXterm deletes your work when you close it. You can see this in the address bar of File Explorer: it shows Temp, for example Local > Temp > Mxt265 > mx86_64b > tmp > home_YourUserName. Before you continue, follow the steps in [MobaXterm: choose a permanent folder](#) on the “Before you start” page. Then make the **data_analysis** folder again.

Next, let’s make a **data** folder inside the new **data_analysis** folder:

```
# Make a data folder inside the data_analysis folder
# The -p option also makes the parent folder (here: data_analysis) if
↳ it does not exist yet
mkdir -p data_analysis/data
```

```
# Check that the folder was created
# Notice how we use ls with an option (-l) and an argument
↪ (data_analysis)?
ls -l data_analysis
```

💡 Tip: Commenting your code

Notice how I added a **#** and some notes above each command?

Anything written after a **#** in bash is a **comment**. A comment is not executed, but it helps you (and others) understand what the code does.

In your own work, add short, meaningful comments above key steps. Avoid restating the obvious. Instead, explain *why* you do something or what it achieves. A good rule: add a comment wherever you think that future you might get confused a month later.

touch: Starting your notebook

Now that we have a project folder, it gets a **notebook**: a file in which you write down the commands you ran, a comment on what each one does, and a few words about what you saw. Think of it as the lab book of your analysis. Writing it while you work takes a bit of time, but it also makes you stop and think about what you just did. And a month from now, it tells you exactly how you got your results.

We create an empty file for the notebook with **touch**:

```
# Create an empty file called notes.md inside the data_analysis
↪ folder
touch data_analysis/notes.md

# Check that the file was created: it has a size of 0, because it is
↪ still empty
ls -l data_analysis
```

touch creates an empty file if it does not exist yet. If the file already exists, **touch** leaves its content alone, so you can not lose your notes by running it twice.

To open the notebook, go to the File Explorer (Windows) or Finder (Mac) window that you opened before, double-click the **data_analysis** folder and then double-click **notes.md**. If it opens in a program other than your text editor, right-click the file, choose **Open with** and pick your editor (for example RStudio).

💡 Tip: RStudio users, stay in Source mode

RStudio shows two buttons at the top left of a Markdown file: **Source** and **Visual**. Stay in **Source**. Then you see exactly what you type, including the # and the backticks.

Writing the notebook in Markdown

The file ends in `.md` because we write it in **Markdown**: plain text with a few symbols that give it structure. For your notebook, you only need three of them:

- A line that starts with # and a space is a **heading**. More # symbols make a smaller heading (`##`, `###`).
- Code goes into a **code block**: a line with three backticks and the word `bash` above the code, and a line with three backticks below it. The backtick (```) is usually at the top left of your keyboard, next to the 1 (on some keyboards you need to press it twice or press the space bar after it).
- Everything else is normal text, for example what you saw or why you did something.

Notice that a # means something different in the two places: in normal Markdown text, it starts a heading. Inside a code block, it is a bash comment.

A first entry in your notebook could look like this:

```
# Notes: Introduction to the command line

## Setting up the project folder

I made a project folder for this workshop in my home directory.

```bash
Go to the home directory and make the project folder with a data
↪ folder inside
cd ~
mkdir data_analysis
mkdir -p data_analysis/data

Make the notebook
touch data_analysis/notes.md
```
```

Markdown can do more, for example lists, links and tables. You don't need these for the workshop, but if you are curious, [Markdown Basics](#) on the Quarto website gives an overview.

From now on, you find a blue notebook box at the end of each part of the tutorial. It tells you what to add to your notebook. For the first few, you can compare your notes with mine. Your notes do not have to look the same as mine, as long as future you understands them.

cd: Move around folders

Now that we have our own project folder, let's learn how to move around the file system with `cd` (short for *change directory*).

The file system is structured like a tree that starts from a single **root** directory, written as a forward slash (/). All other folders branch out from this root directory. For example, we can go from the root directory to the users folder, and from there into the john folder: `/users/john`.

The location of a file or folder is written as a **path**: the folder names along the way, separated by /. There are two ways to write a path, for example to the portfolio folder:

- An **absolute path** starts from the root, e.g. `cd /users/john/portfolio`
- A **relative path** starts from your current location, e.g. `cd portfolio` if you are already in `/users/john`

It is generally recommended to use relative paths from inside your project folder. That makes your code more portable: it still works if you move the project folder or run it on another computer.

Let's practice moving between folders. At each step, use `pwd` if you feel that you get lost:

```
# Move into the data_analysis folder
cd data_analysis

# Check where you are
pwd
```

You now should see something like `/Users/YourUserName/data_analysis`. You can use `cd` in multiple ways to move around:

```
# Move into the data folder
cd data

# Move one level up (..), i.e. go back to the data_analysis folder
cd ..

# Quickly go back home (you now should be in the home directory)
cd ~

# Move multiple levels down at once
cd data_analysis/data

# Move two levels up, i.e. back into the home directory
cd ../..

# And go back to the data_analysis folder
cd data_analysis
```

In the code above, the tilde symbol (~) is a shortcut for your home directory. It is the same as typing the full absolute path to your home (e.g. `cd /Users/YourUserName`), but much faster to type. Two dots (..) always mean “the folder one level up”.

Tip: Command-line completion

Here are some tips for faster navigation (and fewer typos):

- Use the Tab key for autocompletion: type the first few letters of a folder or file name and press Tab. For example, go to your home directory, type `cd data_` and press Tab. The shell fills in `data_analysis` for you.
- If there is more than one match, the shell can not decide which one you mean. Press Tab twice to see all options.
- Use the ↑ / ↓ arrow keys to scroll through commands you ran before. This is useful to rerun a command or to change a small part of it.

From now on, try to use the Tab key often, so that you do not have to type everything yourself.

Exercise

Get familiar with these first commands:

1. Create a new folder called **results** inside your **data_analysis** folder. We will use it later to store the output of our analyses
2. Move into the **results** folder and confirm your location with **pwd**
3. Move back into the **data_analysis** folder
4. Use **ls** to confirm that both **results** and **data** are there

Click me to see an answer

```
# Starting from inside data_analysis
mkdir results
cd results
pwd
cd ..
ls
```

Tip: Finding your Desktop and OneDrive folders (optional)

In this tutorial we stay inside the home directory. If you want to work with files that are somewhere else on your computer, for example on your Desktop, these paths help you find them:

Mac

- Your Desktop is at `/Users/YourUserName/Desktop`, or with the `~` shortcut: `~/Desktop`

MobaXterm and WSL2 (Windows)

The terminal reaches your Windows files via the C drive, which is found at `/mnt/c`:

- Your Desktop is usually at `/mnt/c/Users/YourUserName/Desktop`
- Or at `/mnt/c/Users/YourUserName/OneDrive/Desktop` (when using OneDrive)

Accessing the UvA OneDrive folder

If your OneDrive folder name includes spaces (like **OneDrive - UvA**), put quotes around the path. Otherwise bash does not know what to do with the spaces:

```
cd "/mnt/c/Users/YourUserName/OneDrive - UvA"
```

This is also a good example of why you normally do NOT want spaces in your file and folder names.

Be careful with large sequence files in OneDrive folders: OneDrive tries to upload every file you make there.

Notebook checkpoint: Finding your way

Open `notes.md` and add:

1. The `mkdir` command from the exercise above, which made the `results` folder, with a comment
2. One sentence on the difference between an absolute and a relative path

Save the file.

Compare with my notes

Setting up the project folder

I made a project folder for this workshop in my home
↳ directory.

```
```bash
Go to the home directory and make the project folder
↳ with a data folder inside
cd ~
mkdir data_analysis
mkdir -p data_analysis/data

Make the notebook
touch data_analysis/notes.md

Make a results folder for the output of the analyses
cd data_analysis
mkdir results
```
```

An absolute path starts at the root ``/`` and works from
↳ anywhere.
A relative path starts from the folder I am in, so it
↳ only works from the right place.

wget: Download data

Next, let's download our sequencing data, a file called `seq_project.tar.gz`, with `wget`. Make sure that you are inside your `data_analysis` folder (check with `pwd`), because `wget` saves the file in the folder you are currently in:

```
# Download data using link provided by the sequencing center XYZ on
↳ 23.01.2023
wget
↳ https://raw.githubusercontent.com/ndombrowski/cli_workshop/main/data/seq_project.tar
```

In this example, you can also see how I documented my code: I added where the link came

from and when I got it. This helps future me in case I, for example, need to look for some e-mails about the data later on.

! Wget for Mac users

For Mac users `wget` is not installed by default. If you do not want to install it, you can use the `curl` command instead:

```
# -L: if the server says that the file is somewhere else, follow
→ it until you reach the actual file
# -O: save the file under its original name
curl -LO
→ https://raw.githubusercontent.com/ndombrowski/cli_workshop/main/data/seq_project.ta
```

After you download any kind of data, it is always a good idea to do a sanity check: is the file there, and how large is it? A file size of 0 means that something went wrong during the download.

i Exercise

1. Check with `ls` if the file was downloaded correctly
2. Check the manual if there are extra options that we can use for `ls` to see the file size in an easy-to-read unit, such as K (kilobytes) or M (megabytes) (advanced)

Click me to see an answer

```
# Question 1:
ls -l

# Question 2:
# -h prints the size in a human-readable format
# Options can be combined: -l and -h become -lh
ls -lh
```

cp: Copying data

`cp` (short for *copy*) duplicates files or folders. When working with many files, a good folder structure is essential to keep track of them (useful folders are, for example, `data`, `scripts`

and `results`). To keep our project folder organized, let's copy the `seq_project.tar.gz` file we just downloaded into the `data` folder we made earlier:

```
# Copy the tar.gz file into the data folder
cp seq_project.tar.gz data/

# Check what happened
ls -l
ls -l data
```

When running the two `ls` commands, we see that we now have two copies of `seq_project.tar.gz`: one file is in our working directory and the other one is in our `data` folder. Copying large amounts of data is not ideal, since it uses unnecessary space. However, we can use another command to move the file into our `data` folder instead.

mv: Moving (and renaming) data

`mv` (short for *move*) moves files or folders to a new location without creating a second copy:

```
# Move the tar.gz file into the data folder
mv seq_project.tar.gz data/

# Check what happened
ls -l
ls -l data
```

Notice that `mv` moved the file and, **without asking**, overwrote the copy that we had put into the `data` folder with `cp`. So whenever you use `mv`, make sure that you do not overwrite files by mistake.

`mv` is also the command to rename a file: you “move” it to a new name, e.g. `mv old_name.txt new_name.txt`.

 **Tip:** Downloading directly into a folder

Instead of downloading a file and moving it afterwards, you can tell `wget` directly where to save the file with the `-P` option:

```
wget -P data
```

```
→ https://raw.githubusercontent.com/ndombrowski/cli_workshop/main/data/seq_project.tar.gz
```

MobaXterm users: MobaXterm comes with a simpler version of `wget` that does not know the `-P` option. You see the error `wget: Unknown option P`. In that case, download the file first and then move it with `mv`, as we did above.

Mac users: if you use `curl` instead of `wget`, you can save the file into a specific folder with `--output-dir`:

```
curl --output-dir data -LO
```

```
→ https://raw.githubusercontent.com/ndombrowski/cli_workshop/main/data/seq_project.tar.gz
```

tar: Work with tar files

The data we downloaded is provided as a tar file:

- `tar` is short for Tape Archive, and sometimes referred to as tarball
- The TAR file format is commonly used when storing data
- TAR files are often compressed after being created and then become TGZ files, using the `tgz`, `tar.gz`, or `gz` extension.

We can decompress the data into our data folder as follows:

```
tar -xvf data/seq_project.tar.gz -C data
```

The options we use with the `tar` command are:

- `x` tells tar to extract files from the archive
- `v` (verbose) prints the name of each file while it is being extracted, so you can see what happens
- `f` specifies the file name
- `C` tells tar to change directory (so the package content will be unpacked there)

💡 Tip: how to generate a tarball

If you ever want to generate a tarball you can do the following:

```
tar -cvzf my_tarball.tar.gz folder_to_tar
```

The options we use are:

- **c** create an archive by bundling files and directories together
- **z** use gzip compression when generating the tar file

i Exercise

1. After extracting the tarball, use **ls** to explore the content of the folder we just extracted. Make sure that you also explore the content of the folders inside it.
2. Find the path for at least one sequence file that ends with **fastq.gz**. Hint: use the Tab key (see the tip on command-line completion above), so that you don't have to type every single word.

Click me to see an answer

```
# Question 1:
ls -l data/seq_project

# Question 2:
ls -l data/seq_project/barcode001_User1_ITS_1_L001/
```

rm: Remove files and folders

To remove files and folders, we use the **rm** command (short for *remove*). We want to remove the **seq_project.tar.gz** file, since we do not need it anymore once we have extracted the data:

```
# Remove the tar.gz file from the data folder
rm data/seq_project.tar.gz

# Check if that worked
ls -l data
```

If we want to remove a folder, we need to tell **rm** with an option. For this, we use **-r** (short for *recursive*), which removes a folder together with everything inside it.

! Important

Unix does not have an undelete command.

This means that if you delete something with `rm`, it's gone. There is no recycle bin. Therefore, use `rm` with care and check what you wrote twice before pressing enter! Also, NEVER run `rm -r` or `rm -rf` on the root folder `/` or any other important path, such as your home directory. Always double check the path or file name before you press enter when using `rm`.

Wildcards

After we have explored the folder with our sequencing data with `ls`, we have seen that `data/seq_project` contains 4 folders, 2 folders for two different users who each generated two samples.

You also might have noticed that it gets tedious to figure out how many files are in each folder, because we would need to run `ls` on each single folder and view its contents one by one. Typing every file name by hand also gets tedious, and you risk typos.

Luckily, the shell provides special characters to work with groups of files using pattern matching. These characters are called **wildcards**. The most useful wildcard is `*`, which stands for “any characters, any number of times (also zero times)”. For example, `*.txt` matches all files that end with `.txt`.

We can use `*` to list all files in the `seq_project` folder as follows:

```
ls data/seq_project/
```

```
ls data/seq_project/*
```

will return:

```
data/seq_project/barcode001_User1_ITS_1_L001:  
Sample-DUMMY1_R1.fastq.gz  
Sample-DUMMY1_R2.fastq.gz  
  
data/seq_project/barcode002_User1_ITS_9_L001:  
Sample-DUMMY2_R1.fastq.gz  
Sample-DUMMY2_R2.fastq.gz  
  
data/seq_project/barcode003_User2_ITS_1_L001:  
Sample-DUMMY3_R1.fastq.gz  
Sample-DUMMY3_R2.fastq.gz  
  
data/seq_project/barcode004_User2_ITS_9_L001:  
Sample-DUMMY4_R1.fastq.gz  
Sample-DUMMY4_R2.fastq.gz
```

The `*` matches all characters after `seq_project/` and therefore will list all folders and the content of each folder

You should have seen before that the sequencing files all end with `.fastq.gz`. With this, we can make the command a bit more specific and at the same time make the output more readable. `*/*.gz` means “inside any folder, any file that ends with `.gz`”:

```
ls data/seq_project/*/*.gz
```

```
ls data/seq_project/*/*.gz
```

will return:

```
data/seq_project/barcode001_User1_ITS_1_L001/Sample-DUMMY1_R1.fastq.gz  
data/seq_project/barcode001_User1_ITS_1_L001/Sample-DUMMY1_R2.fastq.gz  
data/seq_project/barcode002_User1_ITS_9_L001/Sample-DUMMY2_R1.fastq.gz  
data/seq_project/barcode002_User1_ITS_9_L001/Sample-DUMMY2_R2.fastq.gz  
data/seq_project/barcode003_User2_ITS_1_L001/Sample-DUMMY3_R1.fastq.gz  
data/seq_project/barcode003_User2_ITS_1_L001/Sample-DUMMY3_R2.fastq.gz  
data/seq_project/barcode004_User2_ITS_9_L001/Sample-DUMMY4_R1.fastq.gz  
data/seq_project/barcode004_User2_ITS_9_L001/Sample-DUMMY4_R2.fastq.gz
```

The first `*` matches all characters between `seq_project/` and the next `/`

The second `*` matches all characters between the last `/` and `.gz`

Now we can easily see for each folder how many files we have.

Exercise

1. Use a wildcard to list files for barcode001 only
2. Use a wildcard to only list R1 (i.e. forward) files
3. Use a wildcard to copy all R1 files into a new folder, call this new folder **R1_files**. Use **ls** to confirm if you copied the right files
4. To avoid duplicates, remove the R1_files folder from question 3 after you have ensured the command you used worked

Click me to see an answer

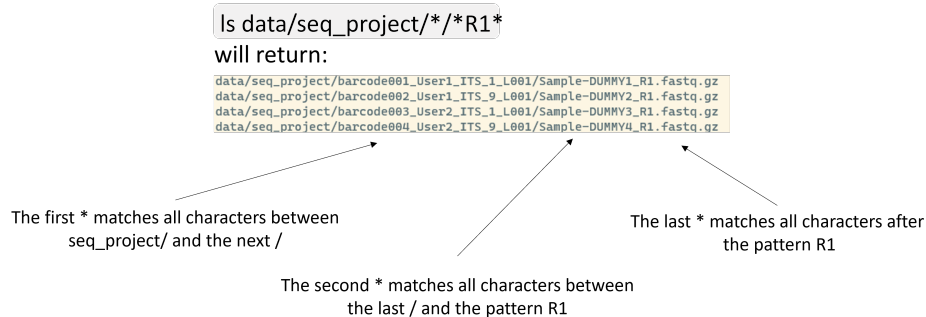
```
# Question 1:
ls data/seq_project/barcode001*/*.gz

# Question 2:
ls data/seq_project/*/R1*

# Question 3:
mkdir R1_files
cp data/seq_project/*/R1* R1_files
ls data/seq_project/*/*.gz
ls -l R1_files

# Question 4:
rm -r R1_files
```

After solving question 2 you should see:



After solving question 3, we should see that all files still exist in their original location but are now also inside the **R1_files** folder. Notice how we can use wildcards together with other commands to easily reorganize

our data?

💡 Tip: More wildcards

`*` is not the only wildcard we can use and a full list can be found [here](#).

Different wildcards can be useful in different contexts, not only when listing files but also finding patterns in files. To make such searches as specific as possible there are different wildcards available. A few examples are:

The `[0-9]` wildcard = matches any number exactly once and allows us to extract a range of files.

```
ls data/seq_project/barcode00[0-9]*/*.gz
```

The `[012]` wildcard = matches 1 or 2 exactly once and allows us to extract a range of files.

```
ls data/seq_project/barcode00[12]*/*.gz
```

`[A-Z]` matches any letter in capitals occurring once

`[a-z]*` matches any letter in non-capital letters occurring many times

```
ls data/seq_project/barcode001_User1_ITS_1_[A-Z]001/*.gz
```

💡 Tip: Finding files with find

With `*/*.gz`, `ls` only looks exactly one folder deep inside `seq_project`. If you don't know how deep your files are stored, you can use `find` instead. `find` searches a folder and all the folders inside it, at any depth:

```
# Search data/seq_project and all its subfolders for files ending  
→ in .gz  
find data/seq_project -name "*.gz"
```

- `data/seq_project` is the folder where `find` starts searching
- `-name "*.gz"` tells `find` to only print files whose name ends with `.gz`

We put quotes around `"*.gz"` so that the shell passes the `*` to `find` unchanged, and `find` uses it as a pattern while it searches.

If you want to see the full (absolute) path of each file, start the search from your home directory with ~:

```
find ~/data_analysis/data/seq_project -name "*.gz"
```

Because the shell replaces ~ with the absolute path to your home directory, **find** now prints absolute paths, and you don't need to type your user name. This works on every system.

Note that **find** does not sort its output alphabetically like **ls** does, so the files may appear in a different order.

i Notebook checkpoint: Getting the data

Add a new heading **## Getting the data** to **notes.md**, and below it:

1. A code block with the commands that got the data into place: download, move, extract, remove the tar file. Leave out the detours, such as the copy we made with **cp** and the **R1_files** folder from the exercise. Someone who runs your code block should end up with the same **data** folder as you
2. In the comment above the download command: where the link came from and on which date you downloaded the data
3. A check that shows what you got, and in normal text below the code block, what it showed: how many folders and how many **.fastq.gz** files
4. Optional: a warning to yourself about the commands that can destroy data without asking (**mv**, **rm**)

Compare with my notes

```

## Getting the data

```bash
Download the data
Link from the sequencing center XYZ, received by
 ↳ email on 23.01.2023
cd ~/data_analysis
wget
 ↳ https://raw.githubusercontent.com/ndombrowski/cli_workshop/main/data/seq_proje

Move the tar file into the data folder and extract it
 ↳ there
mv seq_project.tar.gz data/
tar -xvf data/seq_project.tar.gz -C data

Remove the tar file, we only need the extracted data
rm data/seq_project.tar.gz

Check: list all sequence files
ls data/seq_project/*/*.gz
```

The data has 4 folders, one per sample (2 users with 2
  ↳ samples each).
Each folder has 2 files, R1 and R2, so 8 `.fastq.gz`
  ↳ files in total.

Careful: `mv` overwrites files with the same name
  ↳ without asking,
and files removed with `rm` are gone for good.

```

I/O redirection to new files

What a command prints is called its **standard output**. We have seen that by default, commands print their standard output directly to the terminal. For example, when we use `ls`, the list of files and folders is printed to your terminal. However, we can also **redirect** the standard output into a file by using the `>` character.

For example, we might want to generate a list with all fastq files and can do this as follows:

```
# Write the list of fastq files into a new file instead of to the
↪ screen
ls data/seq_project/*/* > fastq_paths.txt

# Check that the new file exists
ls -l fastq_paths.txt
```

Here:

- `ls data/seq_project/*/*` lists all files in the folders inside `seq_project`
- `>` tells the shell to write this list into a new file called `fastq_paths.txt`, instead of printing it to the screen

! Important: Be careful with `>`

The `>` operator overwrites files **without asking**. If a file with the same name already exists, its content is replaced.

If you want to add (append) to the end of an existing file instead of replacing it, use `>>`.

Exploring the content of text files

Next, let's view the content of the list we just generated. Viewing the actual files we work with is often important to ensure the integrity of our data.

head: View the first rows of a file

`head` can be used to check the first 10 rows of our files:

```
head fastq_paths.txt
```

tail: View the last rows of a file

If you want to check the last 10 rows use `tail`:

```
# Show the last 10 lines
tail fastq_paths.txt

# Show the last 3 lines
# The option -n controls how many lines get printed (this also works
↪ for head)
tail -n 3 fastq_paths.txt
```

less: View the full file

less is a program that lets you view a file's contents one screen at a time. This is useful when dealing with a large text file (such as a sequence data file), because it doesn't load the entire file but accesses it page by page, resulting in fast loading speeds.

```
less -S fastq_paths.txt
```

- You can use the ↑ / ↓ arrow keys and the Page Up / Page Down keys to move through the text file
- To exit less, type q

💡 Tip: Editing text files with **nano**

You can also edit the content of a text file and there are different programs available to do this on the command line, the most commonly used tool is **nano**. It comes with WSL2, Mac and most Linux systems. You can open any file as follows:

```
nano fastq_paths.txt
```

Once the document is open you can edit it however you want and then

- Close the document with **control + X**
- Type y to save changes and press enter

MobaXterm users: nano is not installed in MobaXterm by default, and you see **nano: command not found**. You can install it with **apt install nano**. Like for **wget** above, it is only kept after a restart if you chose a permanent folder in MobaXterm.

wc: Count things

Another useful tool is the `wc` (= wordcount) command that allows us to count the number of lines via `-l` in a file. It is an useful tool for sanity checking and here allows us to count how many files we work with:

```
wc -l fastq_paths.txt
```

After running this we see that we work with a total of 8 files.

We could of course easily count this ourselves, however, if we work with hundreds of files its a quick and easy way to get an overview about how much data we work with.

One down-side of this approach is that to be able to count the number of files, we first need to generate a file in which we count the number of files. This (i) can create files we do not actually need and (ii) we use two commands while ideally we want to get the information with a single command. Pipes, are a way around this problem.

Pipes

So far, we've run one command at a time. But often, you want to combine commands so that the output of one command becomes the input of the next one. That's what the **pipe** (`|`) does. It allows us to chain simple commands together to do more complex things.

Let's use `ls` to list all fastq files and then pipe the output from `ls` into the `wc` command, to count how many files we work with:

```
ls data/seq_project/*/ * | wc -l
```

Here:

- `ls data/seq_project/*/ *` lists all files in the folders inside `seq_project`
- The pipe (`|`) sends this list directly to the next command, instead of printing it to the screen
- `wc -l` counts the lines in this list, i.e. the number of files

We should see again that we work with 8 files, but now we did not have to generate an intermediate text file and have a more condensed command.

Exercise

1. Use `ls` and `wc` to count how many R1 files we have
2. Count how many R2 files we have
3. Count how many files were generated for User1

Click me to see an answer

```
# Question 1: We see 4 files
ls data/seq_project/*/*R1*gz | wc -l

# Question 2: We see 4 files
ls data/seq_project/*/*R2*gz | wc -l

# Question 3: We see 4 files
ls data/seq_project/*_User1_*/*gz | wc -l
```

Checking whether we have the same number of R1 and R2 files is a good sanity check, to ensure that we have received all the files from the sequencing center whenever we generate paired-end sequencing data.

cut: Extract elements from strings

Remember, how we stored a list of sequencing files and the path leading to these files (relative to our home directory in a text file called `fastq_paths.txt`)?

Imagine that we only wanted to have a list of files but not the path, how would we do that?

There are different ways to do this, the simplest one is to use `cd` and go into the folder with our sequence files and generate a list in there.

However, another way in which we do not have to move around (and learn some other concepts) is to use the `cut` command. `cut` allows us to separate lines of text into different elements using any kind of delimiter, for example the `/` that we use in the file path. To ensure that `/` is seen as a separator we use the `-d` option and with `-f4` we tell `cut` to print the fourth element of each separated field.

```
head fastq_paths.txt
```

```
#extract the file name (i.e. the fourth field when using a /  
↪ separator)
```

```
cut -f4 -d "/" fastq_paths.txt
```

```
#do the same as above but this time save the output in a new file
```

```
cut -f4 -d "/" fastq_paths.txt > fastq_files.txt
```

```
cut -d "/"
```

will return the following elements:

```
data/seq_project/barcode001_User1_ITS_1_L001/Sample-DUMMY1_R1.fastq.gz  
data/seq_project/barcode001_User1_ITS_1_L001/Sample-DUMMY1_R2.fastq.gz  
data/seq_project/barcode002_User1_ITS_9_L001/Sample-DUMMY2_R1.fastq.gz  
data/seq_project/barcode002_User1_ITS_9_L001/Sample-DUMMY2_R2.fastq.gz  
data/seq_project/barcode003_User2_ITS_1_L001/Sample-DUMMY3_R1.fastq.gz  
data/seq_project/barcode003_User2_ITS_1_L001/Sample-DUMMY3_R2.fastq.gz  
data/seq_project/barcode004_User2_ITS_9_L001/Sample-DUMMY4_R1.fastq.gz  
data/seq_project/barcode004_User2_ITS_9_L001/Sample-DUMMY4_R2.fastq.gz
```

element1

element2

element3

element4

To extract the file name, we can print only the 4th element with:

```
cut -f4 -d "/"
```

We can also combine this with pipes in order to extract different pieces of information. Let's assume we start with extracting a folder name, such as `barcode002_User1_ITS_9_L001`, and from that we want to extract some other information, such as the barcode IDs. We can easily do this as follows:

```
cut -f3 -d "/" fastq_paths.txt | cut -f1 -d "_"
```

```
cut -f3 -d "/" fastq_paths.txt
```

will return the following:

```
barcode001_User1_ITS_1_L001
barcode001_User1_ITS_1_L001
barcode002_User1_ITS_9_L001
barcode002_User1_ITS_9_L001
barcode003_User2_ITS_1_L001
barcode003_User2_ITS_1_L001
barcode004_User2_ITS_9_L001
barcode004_User2_ITS_9_L001
```

We can further subset this with

```
cut -f1 -d "_"
```

```
barcode001_User1_ITS_1_L001
barcode001_User1_ITS_1_L001
barcode002_User1_ITS_9_L001
barcode002_User1_ITS_9_L001
barcode003_User2_ITS_1_L001
barcode003_User2_ITS_1_L001
barcode004_User2_ITS_9_L001
barcode004_User2_ITS_9_L001
```

element1

element5

To return the barcodeID,
i.e. the first element

Nice, we now only have a list with barcodes. However, its not yet ideal since we have duplicated barcodes. If you want to extract this information for some metadata file this is not ideal and we need to get to know two more commands to make this work:

sort and uniq: Create unique lists

- **sort**: sorts lines in a file from A-Z
- **uniq**: removes or finds duplicates . For this command to work you need to provide it with a sorted file, which is why sort and uniq almost always are used together

Let's first ensure that our barcode list is sorted to then extract only the unique information:

```
cut -f3 -d "/" fastq_paths.txt | cut -f1 -d "_" | sort | uniq
```

i Exercise

1. Use cut on fastq_paths.txt to print the folder names in which the fastq.gz files reside (i.e. barcode001_User1_ITS_1_L001)

2. Print the folder names as in 1 and then extract the user names from this list
3. Ensure that when running the code from 2 that we print a unique list of user names

Click me to see an answer

```
# Question 1:
cut -f3 -d "/" fastq_paths.txt

# Question 2:
cut -f3 -d "/" fastq_paths.txt | cut -f2 -d "_"

# Question 3:
cut -f3 -d "/" fastq_paths.txt | cut -f2 -d "_" | sort
↪ | uniq
```

cat: Read and combine files

cat can do different things:

1. Create new files
2. Display the content of a file
3. Concatenate, i.e. combine, several files. For concatenation we can provide **cat** with multiple file listed after each other or use a wildcard.

To view files we do:

```
cat fastq_paths.txt
```

To combine files we do:

```
# Make a folder for the combined files inside results
# -p: no error if the folder already exists, and also makes results/
↪ if it is missing
mkdir -p results/combined

# Combine two files into one
```

```
cat fastq_paths.txt fastq_files.txt >
  ↪ results/combined/combined_files.txt

# View the new file
cat results/combined/combined_files.txt
```

Notice how we keep the output in its own subfolder of **results**? Giving each analysis its own folder inside **results** keeps the project folder clean, and later you can see right away which files belong together.

Exercise

Fastq.gz files can easily be combined using **cat** and while this is not strictly necessary in our case, you might need to do this with your sequence files in the future. For example if you sequenced a lot of data and for space reasons the sequencing center send you multiple files for a single sample.

1. Using wildcards and **cat**, combine all R1 fastq.gz files into a single file called **combined_R1.fastq.gz**. Store it in the **results/combined** folder
2. Use **ls** to judge the file size of the individual R1 files and the combined file to assess whether everything worked correctly

Click me to see an answer

```
# Question 1:
cat data/seq_project/*/*R1.fastq.gz >
  ↪ results/combined/combined_R1.fastq.gz

# Question 2:
ls -lh data/seq_project/*/*R1.fastq.gz
ls -lh results/combined/combined_R1.fastq.gz
```

If you compare the two numbers generated for question 2, you should see that the **combined_R1.fastq.gz** file is roughly the sum of the individual files.

Exploring the content of compressed fastq files

Exercise

1. View the first few rows of any one of the fastq.gz sequence files
2. Does this look like a normal sequence file to you?

Click me to see an answer

```
head
```

```
↪ data/seq_project/barcode001_User1_ITS_1_L001/Sample-DUMMY1_R1.fastq.gz
```

After running this command you will see a lot of random letters and numbers but nothing that looks like a sequence, so what is going on?

gzip: (De)compress files

You might have noticed that our sequence files end with `.gz`. This extension is used for files that are compressed with a tool called **gzip**. Compressing makes a file smaller, which saves space, but it also makes the file unreadable for a human. That is why `head` showed us random characters.

To view the content of a compressed file, we can decompress it first. We do this with the **gzip** command together with the `-d` (decompress) option. By default, **gzip** compresses a file:

```
# Decompress one file
gzip -d
↪ data/seq_project/barcode001_User1_ITS_1_L001/Sample-DUMMY1_R1.fastq.gz

# Check that this worked: the file lost its .gz extension
ls -l data/seq_project/barcode001_User1_ITS_1_L001/

# View the first lines of the decompressed file
head
↪ data/seq_project/barcode001_User1_ITS_1_L001/Sample-DUMMY1_R1.fastq
```

After running this, we finally see a sequence and some other information. Notice that for fastq files we always should see 4 rows of information for each sequence as shown below:

Exercise

1. After running the `gzip` command above, make a list of each sequence file with `ls`
2. Use `ls` with an option to also view the file size and compare the size of our compressed and uncompressed files.

Click me to see an answer

```
# Question 1:
ls -l data/seq_project/*/*

# Question 2:
ls -lh data/seq_project/*/*
```

We see that the decompressed file has a size of about ~25M, while the compressed files are around ~5M.

When working with sequencing files the data is usually much larger and its best to keep the files compressed to not clutter your computer.

To keep our files small its best to work with the compressed files. If we want to compress a file, we can do this as follows:

```
gzip
↪ data/seq_project/barcode001_User1_ITS_1_L001/Sample-DUMMY1_R1.fastq

ls data/seq_project/*/*
```

zcat: Decompress and print to screen

We have seen that to view the content of a compressed file, we first had to decompress it. This is fine for small files, but sequence files tend to get large, and we might not want to decompress them so that they don't clutter our system.

Luckily, there is a useful tool in bash called **zcat**. It decompresses a file and prints the content to the screen, but leaves the compressed file as it is. We combine the **zcat** command with the **head** command, since we do not want to print millions of sequences to the screen but only want to explore the first few rows:

```
zcat
↪ data/seq_project/barcode001_User1_ITS_1_L001/Sample-DUMMY1_R1.fastq.gz
↪ | head
```

! zcat for Mac users

For Mac users using the zsh shell, `zcat` might not work as expected. Try using `gzcat` instead:

```
gzcat
↪ data/seq_project/barcode001_User1_ITS_1_L001/Sample-DUMMY1_R1.fastq.gz
↪ | head
```

grep: Find patterns in data

The `grep` command is used to search for words, or patterns, in any text file. This command is simple but very useful for sanity checks after file transformations.

The basic syntax is `grep "pattern" file`. Here, “pattern” is the text you want to search for, and file is the file you want to search in. The double quotes allow us to include special characters (like > or spaces) safely.

Let’s first search for something in the text file we generated. For example, we might want to only print information for the R1 files:

```
# Print all lines in fastq_paths.txt that contain R1
grep "R1" fastq_paths.txt
```

We see the list of files that match our pattern. If we simply are interested in the number of files that match our pattern, we could add the option `-c` to count how often the pattern we search for occurs.

```
grep -c "R1" fastq_paths.txt
```

If we want to look only at samples from User1, we can search for that part of the file name:

```
grep "User1" fastq_paths.txt
```

i Exercise

1. Grep for paths containing the pattern “ITS_9” in fastq_paths.txt
2. Grep for the pattern “AAGACG” in any of the fastq.gz files (advanced)
3. Count how often “AAGACG” occurs in any in any of the fastq.gz files (advanced)

Click me to see an answer

```
# Question 1
grep "ITS_9" fastq_paths.txt

# Question 2
zcat
↪ data/seq_project/barcode001_User1_ITS_1_L001/Sample-DUMMY1_R1.fastq.gz
↪ | grep "AAGACG"

# Question 3
zcat
↪ data/seq_project/barcode001_User1_ITS_1_L001/Sample-DUMMY1_R1.fastq.gz
↪ | grep -c "AAGACG"
```

The last two commands can be very useful to check if sequencing adapters or primers are still part of your sequence.

wc: Count things II

We have seen before that `wc` is a command that allows us to count the number of lines in a file and we can easily use it on fastq files to get an idea about how many sequences we work with:

```
zcat
↪ data/seq_project/barcode001_User1_ITS_1_L001/Sample-DUMMY1_R2.fastq.gz
↪ | wc -l
```

After running this we see that we work with 179,384 / 4 sequences. We need to divide the number we see on the screen by 4 since each sequence is represented by 4 lines of

information in our fastq file.

💡 Advanced tip: Better counting

We have seen that we need to divide the output of `wc` by four to get the total number of sequences. We can do this with a calculator but actually, some intermediate bash can also be used to do this for us:

```
zcat
↳ data/seq_project/barcode001_User1_ITS_1_L001/Sample-DUMMY1_R2.fastq.gz
↳ | echo $(( $(wc -l) /4))
```

In the command above we have some new syntax:

- The `echo` command is used to display messages or print information to the terminal. In our case it will print whatever is going on in this section `$(( $(wc -l) /4))`
- `$(...)` indicates a command substitution. It allows the output of the enclosed command (`wc -l`) to be used as part of another command. Without command substitution (`wc -l` alone), you would not capture the output; instead, you would only see the literal text “`wc -l`”.
- `$((...))`: This is an arithmetic expansion in Bash. It allows you to perform arithmetic operations inside the brackets and substitute the result into the command line. In this case, it calculates the result of dividing the number of lines in the fastq file by 4.
- `/4`: This is dividing the result obtained from `wc -l` by 4. Since each sequence in a FASTQ file is represented by four lines (identifier, sequence, separator, and quality scores), dividing the total number of lines by 4 gives the number of sequences.

i Notebook checkpoint: Looking at the data

Add a new heading `## Looking at the data` to `notes.md`, and below it:

1. A code block with the commands that made the two file lists, `fastq_paths.txt` and `fastq_files.txt`
2. In the same code block, the checks that told you something about the data: how many files there are, whether there are as many R1 as R2 files, what the first lines of a fastq file look like, and how many lines one file has
3. In normal text below the code block: what you learned about the data. How

many lines does one read take? Why did `head` on a `.gz` file show strange characters?

4. Leave out the detour where we decompressed a file with `gzip -d` and compressed it again, because it ended with the data as it was. A short note on why we use `zcat` is enough

Then a check that we come back to: run `ls` and `ls results` inside `data_analysis`. Can your notebook explain where every file and folder came from? If not, add a line about it, even if it is “practice file, not needed”.

Compare with my notes

Looking at the data

```
```bash
Make a list of all fastq files with their paths
ls data/seq_project/*/* > fastq_paths.txt

Make a list of the file names only, without the
↳ folder names
cut -f4 -d "/" fastq_paths.txt > fastq_files.txt

Check: we expect 8 files
wc -l fastq_paths.txt

Check: as many R1 as R2 files
ls data/seq_project/*/*R1*.gz | wc -l
ls data/seq_project/*/*R2*.gz | wc -l

Look at the first reads of one file, without
↳ decompressing it
zcat
↳ data/seq_project/barcode001_User1_ITS_1_L001/Sample-DUMMY1_R1.fastq.gz
↳ | head

Count the lines in one file
zcat
↳ data/seq_project/barcode001_User1_ITS_1_L001/Sample-DUMMY1_R2.fastq.gz
↳ | wc -l
```
```

There are 8 files, 4 R1 and 4 R2, so every sample has
↳ both.

The files are compressed with gzip, so ``head`` on a
↳ ``.gz`` file shows unreadable characters.

I use ``zcat`` to look inside: decompressing with ``gzip``
↳ `-d`` makes the files about 5 times larger.

In a fastq file, each read takes 4 lines: identifier,
↳ sequence, separator (``+``) and quality scores.

Sample-DUMMY1_R2 has 179,384 lines, so $179,384 / 4 =$
↳ 44,846 reads.

``results/combined/`` has the practice files from the
↳ ``cat`` section. Not needed for the analysis.

echo and variables: Storing information

Before we learn how we can use the command line to automate repetitive tasks, we need two small building blocks.

The first one is **echo**. It prints the text you give it to the screen:

```
echo "Hello"
```

The second one is a variable. A **variable** is a name that stores a value, for example a word, a number or a file path. You create a variable by writing its name, an equals sign and the value:

```
# Store the path to one sequencing file in a variable called
↪ fastq_file
fastq_file=data/seq_project/barcode001_User1_ITS_1_L001/Sample-DUMMY1_R1.fastq.gz
```

This prints nothing, however Bash will remember the value. To use the value, put a **\$** in front of the variable name. Bash then replaces **\$fastq_file** with the path we stored:

```
# Print the value stored in the variable
echo $fastq_file

# Use the variable in a command, exactly like you would use the path
↪ itself
zcat $fastq_file | wc -l
```

The second command gives the same result as typing out the full path. On its own, that does not save us much work. Variables become really useful when their value changes, and that is exactly what a for-loop does for us in the next section.

A few rules for variables:

- The name can contain letters, numbers and underscores (**_**), but no spaces. Choose a name that tells you what is stored in it, **fastq_file** is easier to understand a year later than **x**.
- A variable only exists in the terminal window where you made it. If you close the terminal, it is gone.

! Important: No spaces around =

Write `fastq_file=data/...` without spaces around the `=`. If you add spaces (`fastq_file = data/...`), bash thinks that `fastq_file` is the name of a command, and you get an error that ends with `fastq_file: command not found`.

For-loops

Imagine we want to count the lines not only in one, but in all files. Could we do something like the code below?

```
zcat data/seq_project/*/*.fastq.gz | wc -l
```

When running this, we see that the command prints a single number, 869 944, but not the counts for each individual file, so something did not work right.

The problem with this command is that it prints the text from all 8 fastq files and only afterwards counts the lines. We basically combined all files and then counted the lines of all 8 files together. However, what we want is to repeat the steps below, individually for each file:

- Decompress a first file
- Count the lines in the first file
- Decompress a second file
- Count the lines in the second file
- ...

A **for-loop** repeats one or more commands, once for each item in a list. Each round of the loop is called an **iteration**.

Step 1: Check what the loop works on

Let's write a first loop that does not change anything yet. It only prints the names of our sequencing files:

```
for fastq_file in data/seq_project/*/*.fastq.gz; do
    echo $fastq_file
done
```

This prints the paths of all 8 fastq files, one per line. Let's go through the parts of the loop:

- `for fastq_file in data/seq_project/**/*.fastq.gz; do`: this line says what to repeat over. The wildcard gives the list, here all 8 fastq files. In each iteration, bash stores the next file from the list in the variable `fastq_file`. You choose the name of this variable yourself.
- `echo $fastq_file`: the command we want to repeat, written between `do` and `done`. Here, we print the value of the variable. We indent this line with 4 spaces. Bash does not need this, but it makes the loop easier to read.
- `done`: marks the end of the loop.

In other words, the loop does the same as typing these commands one after the other:

```
# You don't need to run this, it only shows what the loop does for us
fastq_file=data/seq_project/barcode001_User1_ITS_1_L001/Sample-DUMMY1_R1.fastq.gz
echo $fastq_file
fastq_file=data/seq_project/barcode001_User1_ITS_1_L001/Sample-DUMMY1_R2.fastq.gz
echo $fastq_file
# ... and so on, for the other 6 files
```

Starting a loop with `echo` is a good habit. This is called a **dry run**: the loop shows what it would work on, without changing anything. Before you add the real command, check the list. Do you see all 8 files, and only the files you expect?

Tip: Typing a loop in the terminal

When you press Enter after writing the first line of the loop, nothing runs yet. Instead, the prompt changes to `>`. This means bash waits for the rest of the loop. It only starts once you type `done` and press Enter. (On a Mac with `zsh`, this prompt may look slightly different, for example `for>`.)

If you get stuck in this `>` prompt, for example because of a typo, press `Ctrl+C` to cancel and start again.

Step 2: Add the real command

Now that we know the loop works on the right files, we replace `echo` with the command we actually want to run:

```
for fastq_file in data/seq_project/*/*.fastq.gz; do
    zcat $fastq_file | wc -l
done
```

Now we get 8 numbers, one per file. `zcat $fastq_file | wc -l` is the same command as in the variables section. The only difference is that bash now runs it 8 times, each time with a different file stored in `fastq_file`.

 Tip: Loops written on one line

You might also see loops written on a single line, with `;` where we start a new line:

```
for fastq_file in data/seq_project/*/*.fastq.gz; do zcat
↪ $fastq_file | wc -l; done
```

This does exactly the same as the loop above. For anything longer than one short command, the version over several lines is easier to read.

Exercise

1. Use a for-loop and count how often `grep` finds the pattern “TAAGA” in each individual `fastq.gz` file

Click me to see an answer

```
for fastq_file in data/seq_project/*/*.fastq.gz; do
    zcat $fastq_file | grep -c "TAAGA"
done
```

>>: Collecting the loop output in one file

So far, the counts only appear on the screen. We cannot see which number belongs to which file, and the numbers are gone once we close the terminal. Let’s write the file names and the counts into a file instead.

In the section on I/O redirection, we have seen that `>` replaces the content of a file, and that `>>` adds (**appends**) text to the end of a file. Inside a loop, this difference matters.

With `>`, each iteration would replace what the iteration before it wrote, and only the result for the last file would be left. So inside the loop we use `>>`:

```
# Make a folder for the counts inside results
mkdir -p results/counts

# Start the counts file with a first line that describes its content
# Because we use > here, an existing file is emptied first, so we
↪ start fresh each time
echo "Number of lines per fastq file" >
↪ results/counts/line_counts.txt

# For each fastq file, add the file name and its number of lines to
↪ the end of the file
for fastq_file in data/seq_project/*/*.fastq.gz; do
    echo $fastq_file >> results/counts/line_counts.txt
    zcat $fastq_file | wc -l >> results/counts/line_counts.txt
done

# Check the result: we expect 1 header line + 2 lines for each of the
↪ 8 files = 17 lines
wc -l results/counts/line_counts.txt
cat results/counts/line_counts.txt
```

Here:

- `mkdir -p results/counts`: we keep the output of each analysis in its own folder inside `results`
- `echo "Number of lines per fastq file" > results/counts/line_counts.txt`: before the loop starts, we write a first line that describes what is in the file. Such a line is called a **header**. Because we use `>` here, this also empties the file if it already exists from an earlier run
- Inside the loop, there are now two commands. The first one adds the name of the file, the second one adds its number of lines. Both use `>>`, so each iteration adds two lines to the end of `line_counts.txt`

We will use these numbers again on day 2, to check what a quality filtering tool did to our data.

Exercise

1. Run only the for-loop from the code above a second time, without the `echo` line before it. How many lines does `line_counts.txt` have now? Why?
2. Now run the complete code again, including the `echo` line before the loop. How many lines does `line_counts.txt` have now?
3. What do you think would happen if we used `>` instead of `>>` inside the loop? Think about it first, then try it. Afterwards, run the code with `>>` once more, so that `line_counts.txt` is correct again.

Click me to see an answer

1. The file now has 33 lines. The loop added its 16 lines a second time to the end of the file, so all counts appear twice. This is something to watch out for whenever you use `>>`.
2. The file has 17 lines again. The `echo` line uses `>`, which empties the file before the loop adds the counts.
3. Each `>` replaces the content of the file. After the loop, the file only contains a single line: the number of lines of the last file. All other results were overwritten.

Advanced tip: Better counting in for loops

Let's get a bit more advanced to show you some powerful features of bash. Above, the file name and its count end up on two separate lines. Imagine that you would do this for 100 files. In this case it would be useful to see each file name and its count on the same line. We can achieve this with the command substitution from the **Advanced tip: Better counting** in the `wc` section above.

```
for fastq_file in data/seq_project/*/*.fastq.gz; do
    echo "$fastq_file: $(zcat $fastq_file | wc -l)"
done
```

- The `echo` command is used to display messages or print information to the terminal. In our case it will print whatever is going on here `"$fastq_file: $(zcat $fastq_file | wc -l)"`
- `$(...)`: These parentheses are used for command substitution. It means that the command `zcat $fastq_file | wc -l` is executed, and its output (the line count of the uncompressed content) is substituted in that position.
- The double quotes (`"`) perform what is called a string concatenation. It

concatenates the filename (`$fastq_file`), a colon (:), a space, and the line count obtained from the command substitution. The entire string is then passed as a single argument to the `echo` command.

Almost perfect, now we only want to divide this by 4:

```
for fastq_file in data/seq_project/*/*.fastq.gz; do
    echo "$fastq_file: $(( $(zcat $fastq_file | wc -l) /4 ))"
done
```

Notice, how we use both single brackets and double brackets?

- `$((...))` in contrast to `$(...)` is an arithmetic expansion in Bash. It allows you to perform arithmetic operations and substitute the result into the command line.

💡 Advanced tip: Using bash to make sample mapping files

For a lot of mapping files for amplicon analyses, you need to prepare a table that lists your files, often with the R1 and R2 files in separate columns. How could we do this using bash?

This code is more advanced and uses a few concepts we have not talked about but should give you an idea about the different data transformations you can do with bash.

```
for file in data/seq_project/*/*.gz; do
    if [[ $file == *_R1.fastq.gz ]]; then
        r1_file=$file
        r2_file=$(echo $file | sed 's/_R1/_R2/')
        echo "$r1_file,$r2_file"
    fi
done
```

1. `for file in data/seq_project/*/*.gz`: We loop through all `fastq.gz` files
2. `if [[ $file == *_R1.fastq.gz ]]; then`: This line checks if the current file's name matches the pattern `*_R1.fastq.gz`. The `[[ ... ]]` is a conditional construct in bash, and the `==` is used for string comparison. If the condition is true (i.e., the file is an R1 file), then the code inside the `if` block will be executed. The block starts with `if` and finishes at the `fi`.

3. `r1_file=$file`: This line involves creating a variable named `r1_file` and assigning it the value of the current file (`$file`). A variable is a symbolic name or identifier associated with a value or data. It allows you to store and retrieve information for later use. Here, `r1_file` is used to store the name of the current R1 file.
4. `r2_file=$(echo $file | sed 's/_R1/_R2/')`: This line sets the variable `r2_file` by replacing `_R1` with `_R2` in the current file name. This is done using the `sed` command, which is a stream editor for filtering and transforming text.
5. `echo "$r1_file,$r2_file"`: This line prints the R1 and R2 file names separated by a comma. This will be part of the output in the format `R1,R2`.

We can do even more and add a column with the sample name with what we have learned before:

```
for file in data/seq_project/*/*.gz; do
    if [[ $file == *_R1.fastq.gz ]]; then
        sample_name=$(echo $file | cut -f3 -d "/" )
        r1_file=$file
        r2_file=$(echo $file | sed 's/_R1/_R2/')
        echo "$sample_name,$r1_file,$r2_file"
    fi
done
```

You can easily combine this with some of the things we learned, such as using `cut` to extract specific parts of the `sample_name` as well. I.e. lets assume you only want to list the index:

```
for file in data/seq_project/*/*.gz; do
    if [[ $file == *_R1.fastq.gz ]]; then
        sample_name=$(echo $file | cut -f3 -d "/" |cut -f1 -d
↵  "_" )
        r1_file=$file
        r2_file=$(echo $file | sed 's/_R1/_R2/')
        echo "$sample_name,$r1_file,$r2_file"
    fi
done
```

bash: Running commands from a script

Our counting loop is now a few lines long, and we might want to run it again later, for example when new data arrives. Instead of typing or copying the loop each time, we can save it in a file. A text file that contains commands is called a **script**. Bash scripts usually end with `.sh`.

To keep our project folder tidy, we store scripts in their own folder:

```
# Make a folder for scripts inside data_analysis
mkdir -p scripts

# Check that the folder was created
ls -l
```

Next, we create the script with the text editor `nano` (see the tip “Editing text files with `nano`” above):

```
nano scripts/count_lines.sh
```

MobaXterm users: if you see `nano: command not found`, install it first with `apt install nano`, as explained in that tip.

Type or copy the following into the editor:

```
#!/bin/bash

# Count the number of lines in each fastq file
# Run this script from inside the data_analysis folder

# Make a folder for the counts and start the counts file with a
↪ header
mkdir -p results/counts
echo "Number of lines per fastq file" >
↪ results/counts/line_counts_bash.txt

# For each fastq file, add the file name and its number of lines to
↪ the end of the file
for fastq_file in data/seq_project/*/*.fastq.gz; do
    echo $fastq_file >> results/counts/line_counts_bash.txt
```

```
zcat $fastq_file | wc -l >> results/counts/line_counts_bash.txt
done
```

Save and exit:

- Press **Ctrl+X** to exit, then **Y** to confirm that you want to save the file, then **Enter** to keep the file name

The commands are the same as in the `>>` section. Two things are new:

- `#!/bin/bash`: this first line is called the **shebang**. It tells the system that the script should be run with bash. Put it at the very top of every bash script.
- The lines that start with `#` are comments. Bash ignores them, but they tell you (and others) what the script does and how to use it.

Now we can run the whole script with a single command:

```
# Run the script with bash
bash scripts/count_lines.sh

# Check the output: we again expect 17 lines
wc -l results/counts/line_counts_bash.txt
cat results/counts/line_counts_bash.txt
```

The script itself prints nothing to the screen, because all output goes into `line_counts_bash.txt`. That is why we check the file afterwards.

Why is this useful?

- You can run the same analysis again, with exactly the same commands, without typing mistakes.
- The script documents what you did. Together with your notebook (`notes.md`), it tells future you how a result was made.
- On day 2, we use exactly this kind of script on the HPC. There, we add a few extra lines at the top that tell the HPC how many resources our job needs.

! Important: Write scripts in nano or a code editor, not in Word or Notepad

Windows programs such as Notepad end every line with an extra invisible character. Bash on Linux does not understand this character, and the script fails with errors such as `$'\r': command not found` or `syntax error near unexpected token`. Word

processors such as Word also add formatting and can change straight quotes (") into curly quotes.

To avoid this, write your scripts with `nano`. If you prefer a code editor such as VS Code, check that it saves the file with LF line endings (in VS Code, you see LF or CRLF in the bar at the bottom right, and can click it to change it). This becomes important on day 2, where the scripts run on a Linux computer.

i Notebook checkpoint: Counting lines per file

Add a new heading `## Counting lines per file` to `notes.md`, and below it:

1. The command that ran the script, with a comment that says what the script does and which file it makes. We use `results/counts/line_counts_bash.txt` again on day 2
2. The check you ran afterwards, and below the code block, what you expected and what you saw
3. In your own words: what happens when you run a loop with `>>` twice, and how the `echo` line with `>` before the loop prevents this

You don't need to copy the loop itself into your notebook, because it is saved in `scripts/count_lines.sh`. The script is part of your documentation. Your notebook only needs to say which script you ran and what it made.

Wrapping up day 1

You now have a project folder with data, results, a script, and a notebook that explains how they got there. Before you close the terminal, take a few minutes to read through your notebook.

i Notebook checkpoint: Tidy your notebook

1. Below the main heading at the top of `notes.md`, add 2 or 3 sentences: what data this is, where it came from, and what you did with it today
2. Read your notebook from the top. Could you redo today's analysis from it alone, without this tutorial? Add what is missing
3. Check once more that your notebook explains every file and folder in your project. Run these inside `data_analysis`: `ls`, `ls results/*` and `ls scripts`
4. Add a heading `## Questions for day 2` and write down anything that is still

unclear

On day 2, you keep writing in the same `notes.md` on your own computer, also for the commands that you run on the HPC.